Fiche 15
Algorithmique et Python
Mathématiques ▪ Première spécialité

Exercice 1 • Lire un programme : suite récurrente et seuil

On considère le programme Python suivant.

```
u = 5
n = 0
while u - 2 >= 0.01:
    u = u / 2 + 1
    n = n + 1
print(n, u)
```

1. Quelle suite (u_n) ce programme calcule-t-il ? Préciser son premier terme et sa relation de récurrence.
2. Calculer les cinq premières valeurs prises par la variable `u`.
3. Quelle condition provoque l'arrêt de la boucle ? Que représente la valeur de `n` affichée ?
4. On admet que, pour tout entier naturel n, $u_n = 2 + \dfrac{3}{2^n}$. Déterminer la valeur de `n` affichée par le programme.

Exercice 2 • Compléter un programme : doublement d'un capital

Un capital u_0 est placé à intérêts composés au taux annuel de `taux` %. La fonction suivante doit renvoyer le nombre d'années nécessaires pour que le capital double.

```
def doublement(u0, taux):
    u = u0
    n = 0
    while u < 2 * u0:
        u = ..........
        n = n + 1
    return n
```

1. Recopier et compléter la ligne manquante.
2. Que renvoie l'appel `doublement(1000, 3)` ? On donne $1{,}03^{23} \approx 1{,}974$ et $1{,}03^{24} \approx 2{,}033$.
3. La « règle des 72 » affirme qu'un capital placé au taux de t % double en environ $\dfrac{72}{t}$ années. Ce résultat est-il cohérent avec celui de la question 2 ?
4. Modifier la fonction pour qu'elle renvoie aussi la valeur finale du capital.

poussemaths.fr

Exercice 3 • Écrire une fonction : suite de Fibonacci

Dans cet exercice, la suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$ et, pour tout entier naturel n, $F_{n+2} = F_n + F_{n+1}$.

1. Écrire une fonction Python `fibo(n)` qui renvoie F_n.
2. Écrire une fonction `liste_fibo(n)` qui renvoie la liste $[F_0, F_1, \ldots, F_n]$.
3. Calculer « à la main » la valeur de `fibo(10)`.
4. Écrire des instructions qui affichent les quotients $\dfrac{F_{n+1}}{F_n}$ pour n allant de 5 à 9. Que constate-t-on ?
   (voir l'exercice 7 de la fiche 1)

Exercice 4 • Algorithme de Syracuse

On considère la fonction Python suivante, où `n` est un entier naturel non nul.

```
def syracuse(n):
    suite = [n]
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        suite.append(n)
    return len(suite)
```

1. Exécuter « à la main » cette fonction pour $n = 6$, en donnant la liste des valeurs successives de `n`.
2. Que renvoie l'appel `syracuse(6)` ?
3. Expliquer le rôle de l'instruction `suite.append(n)`.
4. Modifier la fonction pour qu'elle renvoie la plus grande valeur atteinte par `n`.

Exercice 5 • Simulation : approximation de π

On rappelle que `random.uniform(-1, 1)` renvoie un nombre réel aléatoire de l'intervalle $[-1\,;\,1]$.

```
import random
def approx_pi(n):
    dedans = 0
    for i in range(n):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)
        if x**2 + y**2 <= 1:
            dedans = dedans + 1
    return 4 * dedans / n
```

1. Le point $M(x\,;y)$ est choisi au hasard dans le carré de côté 2 centré à l'origine. Quelle est l'aire de ce carré ? Celle du disque de centre O et de rayon 1 ?
2. En déduire la probabilité que $x^2 + y^2 \leqslant 1$, puis expliquer pourquoi on multiplie par 4 à la dernière ligne.
3. Quelle valeur approximative l'appel `approx_pi(1000000)` renvoie-t-il ?
4. Comment améliorer la précision de l'approximation ? Sur quel phénomène repose cette méthode ?

poussemaths.fr

Exercice 6 • Algorithme de seuil et probabilités

On considère la fonction Python suivante, où `p` et `s` sont deux réels de $]0\,;\,1[$.

```
def seuil(p, s):
    n = 0
    proba_aucun = 1
    while proba_aucun > s:
        proba_aucun = proba_aucun * (1 - p)
        n = n + 1
    return n
```

1. On répète des épreuves indépendantes dont la probabilité de succès est `p`. Que représente la valeur de `proba_aucun` après k passages dans la boucle ?
2. Que renvoie l'appel `seuil(0.15, 0.05)` ? *(on pourra utiliser l'exercice 6 de la fiche 11)*
3. Interpréter ce résultat en termes de probabilité d'obtenir au moins un succès.
4. Modifier la fonction pour qu'elle affiche la valeur de `proba_aucun` à chaque étape.

Exercice 7 • Calculer une somme avec une boucle

1. Écrire une fonction `somme_arith(u0, r, n)` qui renvoie la somme $u_0 + u_1 + \cdots + u_n$ des termes d'une suite arithmétique de premier terme `u0` et de raison `r`.
2. Écrire une fonction `somme_geo(v0, q, n)` qui renvoie la somme $v_0 + v_1 + \cdots + v_n$ des termes d'une suite géométrique de premier terme `v0` et de raison `q`.
3. Quelles valeurs renvoient `somme_arith(1, 2, 9)` et `somme_geo(1, 2, 9)` ?
4. Vérifier ces deux résultats à l'aide des formules du cours.

Exercice 8 • Questions à réponse courte

Pour chacune des questions suivantes, donner directement la réponse, sans justification. Aucune proposition de réponse n'est fournie.

1. Après l'exécution de `L = [1, 2, 3]` puis `L.append(4)`, que contient la liste `L` ?
2. Quelle liste est obtenue par l'instruction `list(range(2, 10, 3))` ?
3. Quelle est la valeur de l'expression `13 // 4` ?
4. Quelle est la valeur de l'expression `13 % 4` ?

Exercice 9 • Vrai ou faux

Pour chacune des affirmations suivantes, indiquer si elle est vraie ou fausse, puis justifier la réponse. Pour montrer qu'une affirmation est fausse, un contre-exemple suffit.

Affirmation 1. La boucle `while True:` ne s'arrête jamais, sauf si elle contient une instruction `break` ou `return`.

Affirmation 2. Une liste Python peut contenir des éléments de types différents.

Affirmation 3. Toute fonction Python doit obligatoirement contenir l'instruction `return`.

Affirmation 4. L'expression `10 % 3 == 1` a pour valeur `True`.

poussemaths.fr